

Rapport de travaux pratiques

Module NoSQL Security

Séance 1 — Découvrir NoSQL et prise en main de MongoDB

Objet	Synthèse du travail réalisé et réponses au devoir final
Date de rédaction	8 mars 2026

Table des matières

1	Objet du rapport	2
2	Vue d'ensemble du travail réalisé	2
3	Déroulement technique des travaux	2
3.1	Démarrage initial et premières erreurs	2
3.2	Lancement fonctionnel du conteneur de travail	2
3.3	Connexion à MongoDB et constats de sécurité au démarrage	3
3.4	Jeu de données et opérations CRUD	3
3.5	Vérification de la surface d'attaque	3
4	Réponses au devoir maison	3
4.1	Partie A — Questions de cours	3
4.2	Partie B — Pratique MongoDB	4
5	Évaluation synthétique du travail réalisé	9
6	Conclusion	10

1. Objet du rapport

Ce rapport présente le travail effectivement réalisé dans le laboratoire MongoDB décrit dans le support de cours. Il reprend les étapes techniques observées dans le fichier `Mongodb-Commands.txt`, structure les résultats de manière professionnelle et répond à la dernière partie du support, à savoir le *devoir maison* composé d'une partie théorique et d'une partie pratique.

Le rapport s'appuie sur les commandes réellement exécutées et sur leurs sorties observées. Lorsqu'une action réalisée diffère légèrement de l'énoncé attendu, cette différence est explicitement signalée afin de conserver une traçabilité technique correcte.

2. Vue d'ensemble du travail réalisé

Le travail mené couvre quatre axes principaux :

- 1) **Mise en place du lab MongoDB sous Docker** : téléchargement des images, lancement de conteneurs, diagnostic des erreurs de syntaxe et connexion à `mongosh`.
- 2) **Manipulation des données de sécurité** : insertion, lecture, mise à jour et suppression de documents dans la collection `security_logs`.
- 3) **Requêtes d'investigation** : filtrage par sévérité, projection de champs, tri chronologique et vérification des modifications.
- 4) **Constat de la surface d'attaque** : validation de l'exposition du port 27017, absence d'authentification, et confirmation du bind réseau ouvert.

3. Déroulement technique des travaux

3.1 Démarrage initial et premières erreurs

Le journal d'exécution montre une première tentative de démarrage avec l'image `mongodb/mongodb-community`. Le conteneur a bien été lancé, mais deux difficultés sont apparues :

- `mongosh` n'était pas disponible directement dans le terminal Windows hôte ;
- des commandes MongoDB ont été saisies dans `cmd` au lieu d'être exécutées depuis `mongosh`, ce qui a provoqué des erreurs de type `not recognized as an internal or external command`.

Une seconde difficulté a concerné l'utilisation d'une syntaxe multi-ligne de style Linux avec des antislashes `\` dans `cmd.exe`, ce qui a produit l'erreur `docker: invalid reference format`. La correction a consisté à reformuler la commande sur une seule ligne.

3.2 Lancement fonctionnel du conteneur de travail

La commande opérationnelle de déploiement a été la suivante :

```
docker run -d --name mongo-noseclab -p 27017:27017 -v mongo-data:/data/db mongo:7
```

Cette exécution a permis :

- de télécharger l'image `mongo:7` si elle n'était pas présente localement ;
- d'exposer le port `27017/tcp` sur l'hôte ;
- de monter un volume Docker persistant ;
- de lancer le conteneur `mongo-noseclab`.

Le résultat observé dans `docker ps` confirme que le conteneur est actif et que le port est publié sur `0.0.0.0:27017` et `[::]:27017`. Cela établit déjà une première conclusion sécurité : le service n'est pas limité à `localhost`.

3.3 Connexion à MongoDB et constats de sécurité au démarrage

La connexion à MongoDB via `docker exec -it mongo-noseclab mongosh` a réussi. Les messages de démarrage mettent explicitement en évidence le warning suivant :

```
Access control is not enabled for the database. Read and write access to data
and configuration is unrestricted
```

Ce message est central : il prouve que l'instance a été démarrée sans authentification activée. Une simple commande de test, `db.runCommand({ping:1})`, retourne `{ ok: 1 }`, ce qui confirme le bon fonctionnement du serveur.

3.4 Jeu de données et opérations CRUD

Le journal montre l'insertion du dataset initial `security_logs`, puis plusieurs opérations de manipulation :

- insertion d'un document supplémentaire `eve` ;
- insertion de trois nouveaux documents correspondant à l'énoncé du devoir ;
- requête de sélection sur les événements à sévérité élevée ;
- mise à jour de tous les événements issus de l'IP `10.0.0.50` ;
- suppression d'un document de déconnexion `alice`.

Deux erreurs de syntaxe sont visibles avant l'insertion finale des trois documents demandés. Elles proviennent d'une parenthèse ou d'un crochet mal positionné dans `insertMany`. Après correction, l'insertion est bien confirmée par `acknowledged: true` avec trois `ObjectId` générés.

3.5 Vérification de la surface d'attaque

Le support de cours impose trois contrôles de sécurité principaux, qui ont tous été vérifiés en pratique :

- Port exposé** : le scan `nmap -p 27017 localhost` montre `27017/tcp open mongod`.
- Service joignable** : `curl -s http://localhost:27017` retourne le message MongoDB indiquant qu'un accès HTTP est tenté sur le port natif du driver.
- Absence d'authentification** : `db.adminCommand({ listDatabases: 1 })` renvoie la liste complète des bases, sans aucun identifiant.
- Bind réseau** : `docker port mongo-noseclab` indique `27017/tcp -> 0.0.0.0:27017`.

Ces résultats démontrent que le service est exposé et que les données sont accessibles sans contrôle d'accès.

4. Réponses au devoir maison

4.1 Partie A — Questions de cours

A1. Pourquoi MongoDB est particulièrement adapté aux logs de sécurité hétérogènes

MongoDB est bien adapté au stockage de logs de sécurité parce qu'il repose sur un modèle document flexible. Les événements provenant d'un pare-feu, d'un système d'authentification, d'un antivirus ou d'un scanner réseau ne possèdent pas forcément les mêmes champs. Une base relationnelle impose en général un schéma plus rigide, alors que MongoDB permet à chaque document d'avoir sa propre structure. Cela évite de multiplier les tables ou d'ajouter de nombreuses colonnes inutiles. Ce modèle facilite aussi l'ingestion rapide de grands volumes de logs et permet des requêtes ciblées sur des attributs comme l'IP source, l'action, la sévérité ou l'horodatage.

A2. Différence entre `updateOne(...,{ role: "admin" })` et `updateOne(...,{ $set: { role: "admin" } })`

La commande `updateOne({ user: "alice" }, { role: "admin" })` remplace le document entier par un nouveau document ne contenant que le champ `role` (en dehors de `_id`). Elle est donc dangereuse, car elle détruit les autres champs existants. En revanche, `updateOne({ user: "alice" }, { $set: { role: "admin" } })` modifie uniquement le champ ciblé tout en conservant le reste du document. La version dangereuse est donc celle sans `$set`, car elle provoque une perte de données.

A3. Les trois piliers du triptyque de sécurité appliqué à MongoDB

Les trois piliers sont :

- 1) **Réseau** : déterminer qui peut atteindre le service. Exemple MongoDB : publier le port uniquement sur `127.0.0.1` au lieu de `0.0.0.0`.
- 2) **Authentification** : déterminer qui est l'utilisateur. Exemple MongoDB : activer `-auth` et créer des comptes d'administration ou de service.
- 3) **Autorisation** : déterminer ce que l'utilisateur authentifié peut faire. Exemple MongoDB : attribuer un rôle en lecture seule à un analyste au lieu de lui donner des droits d'écriture ou d'administration.

A4. Qu'est-ce qu'un `ObjectId`

Un `ObjectId` est un identifiant unique généré par MongoDB pour le champ `_id`. Il est composé de 12 octets et est généralement représenté sous forme hexadécimale sur 24 caractères. Il encode notamment un horodatage de création ainsi que d'autres informations internes servant à garantir l'unicité de l'identifiant, comme des composantes liées au processus et à un compteur.

A5. Pourquoi `-p 27017:27017` est plus dangereux que `-p 127.0.0.1:27017:27017`

Avec `-p 27017:27017`, Docker expose le service sur toutes les interfaces réseau de la machine hôte. Toute machine capable d'atteindre cette IP peut donc tenter une connexion au service MongoDB. Avec `-p 127.0.0.1:27017:27017`, l'accès est limité à la machine locale. Cette seconde configuration réduit fortement la surface d'attaque et empêche un accès direct depuis le réseau local ou Internet.

4.2 Partie B — Pratique MongoDB

B1. Insertion de trois nouveaux documents

Après deux tentatives contenant des erreurs de syntaxe, l'insertion correcte réalisée est la suivante :

```
db.security_logs.insertMany([
  src_ip:'10.10.10.10', action:'SSH_Connection', severity:'high', timestamp: new Date(),
  device:'server-db02', user:'admin', severity:'critical', action:'malware', timestamp: new Date(),
  user:'alice', severity:'info', action:'logout', timestamp: new Date()
])
```

Résultat observé :

```
acknowledged: true, insertedIds:      '0': ObjectId('69a96bb49bbcd659ad8563be'),      '1': ObjectId('69a96bb49bbcd659ad8563be')
```

Analyse. L'opération répond globalement à l'énoncé : trois événements ont bien été ajoutés, avec les sévérités demandées et des horodatages générés à l'instant de l'insertion. En revanche, les champs ne sont pas strictement homogènes avec la structure du dataset initial : le premier document ne précise pas explicitement un statut d'échec SSH et le troisième document ne précise ni `src_ip` ni `status`. Le travail reste valide comme démonstration CRUD, mais une modélisation plus rigoureuse serait recommandée.

```
noseclab> db.security_logs.insertMany([ {src_ip:'10.10.10.10', action:'SSH_Connection', severity:'high',timestamp: new Date()}, {device:'server-db02',
user:'admin', severity:'critical', action:'malware',timestamp: new Date()}, {user:'alice', severity:'info', action:'logout',timestamp: new Date()}])
{ acknowledged: true,
  insertedIds: {
    '0': ObjectId('69a96bb49bbd659ad8563be'),
    '1': ObjectId('69a96bb49bbd659ad8563bf'),
    '2': ObjectId('69a96bb49bbd659ad8563c0')
  }
}
```

FIGURE 1 – Insertion réussie des trois documents demandés.

B2. Requête des événements high ou critical après 10h00

La commande exécutée et fonctionnelle est :

```
db.security_logs.find(
  severity: $in:["high","critical"], timestamp: $gte: ISODate("2026-03-04T10:00:00.000Z"),
  timestamp:1, src_ip:1, action:1, severity:1, _id:0
).sort( timestamp: -1 )
```

Résultat observé :

```
[
  {
    src_ip: '10.10.10.10',    action: 'SSH_Connection',    severity: 'high',    timestamp: ISODate('2026-03-05T11:40:36.154Z')
    severity: 'critical',    action: 'malware',    timestamp: ISODate('2026-03-05T11:40:36.154Z')
  }
]
```

Analyse. La logique de filtre, de projection et de tri est correcte. Le seuil temporel utilisé est cependant 2026-03-04T10:00:00.000Z, alors que l'énoncé demandait conceptuellement les événements après 10h00 UTC. Comme les trois nouveaux documents ont été créés le 5 mars, la requête retourne bien les documents insérés récemment avec les sévérités attendues. Le troisième document n'apparaît pas car sa sévérité est `info`.

```
noseclab> db.security_logs.find({severity:{$in:["high","critical"]},timestamp:{$gte:ISODate("2026-03-04T10:00:00.000Z")}},
{timestamp:1, src_ip:1, action:1, severity:1, _id:0}).sort({ timestamp: -1 })
[
  {
    src_ip: '10.10.10.10',
    action: 'SSH_Connection',
    severity: 'high',
    timestamp: ISODate('2026-03-05T11:40:36.154Z')
  },
  {
    severity: 'critical',
    action: 'malware',
    timestamp: ISODate('2026-03-05T11:40:36.154Z')
  }
]
```

FIGURE 2 – Requête filtrant les événements high ou critical avec projection et tri décroissant.

B3. Mise à jour des documents issus de l'IP 10.0.0.50

La commande exécutée est :

```
db.security_logs.updateMany(  
  src_ip:"10.0.0.50",  
  $set:investigate: true, analyst:"Mohammed_Z"  
)
```

Résultat observé :

```
acknowledged: true, insertedId: null, matchedCount: 4, modifiedCount: 4, upsertedCount: 0
```

Vérification effectuée ensuite :

```
db.security_logs.find(src_ip : "10.0.0.50")
```

Le résultat montre bien quatre documents modifiés, chacun portant les champs `analyst: 'Mohammed_Z'` et `investigate: true`.

Analyse. L'intention du devoir est respectée : tous les documents liés à l'IP ciblée ont été enrichis d'un marqueur d'investigation et d'un analyste. En revanche, deux écarts mineurs existent par rapport à l'énoncé :

- le champ créé est `investigate` et non `investigated`;
- la valeur de l'analyste est `"Mohammed_Z"` au lieu du placeholder `"votre_nom"`.

Sur le plan technique, la mise à jour est néanmoins valide et démontrée.

```
noseclab> db.security_logs.updateMany({src_ip:"10.0.0.50"},{$set:{investigate: true,analyst:"Mohammed_Z"}})  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 4,  
  modifiedCount: 4,  
  upsertedCount: 0  
}
```

FIGURE 3 – Résultat de `updateMany` sur l'IP 10.0.0.50.

```
noseclab> db.security_logs.find({src_ip : "10.0.0.50"})
[
  {
    _id: ObjectId('69a9591c9bbbed659ad8563b3'),
    timestamp: ISODate('2026-02-25T08:12:45.000Z'),
    src_ip: '10.0.0.50',
    user: 'bob',
    action: 'login',
    status: 'failure',
    device: 'desktop',
    geo: { country: 'FR', city: 'Lyon' },
    severity: 'high',
    analyst: 'Mohammed_Z',
    investigate: true
  },
  {
    _id: ObjectId('69a9591c9bbbed659ad8563b4'),
    timestamp: ISODate('2026-02-25T08:13:10.000Z'),
    src_ip: '10.0.0.50',
    user: 'bob',
    action: 'login',
    status: 'failure',
    device: 'desktop',
    geo: { country: 'FR', city: 'Lyon' },
    severity: 'high',
    analyst: 'Mohammed_Z',
    investigate: true
  },
  {
    _id: ObjectId('69a9591c9bbbed659ad8563b5'),
    timestamp: ISODate('2026-02-25T08:13:42.000Z'),
    src_ip: '10.0.0.50',
    user: 'bob',
    action: 'login',
    status: 'failure',
    device: 'desktop',
    geo: { country: 'FR', city: 'Lyon' },
    severity: 'high',
    analyst: 'Mohammed_Z',
    investigate: true
  },
  {
    _id: ObjectId('69a9591c9bbbed659ad8563b6'),
    timestamp: ISODate('2026-02-25T08:15:00.000Z'),
    src_ip: '10.0.0.50',
    user: 'bob',
    action: 'login',
    status: 'success',
    device: 'desktop',
    geo: { country: 'FR', city: 'Lyon' },
    severity: 'info',
    analyst: 'Mohammed_Z',
    investigate: true
  }
]
```


B4. Suppression du document de déconnexion d'Alice et contrôle du nombre total

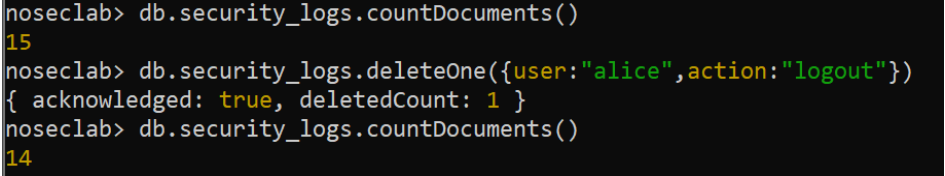
Les commandes observées sont :

```
db.security_logs.countDocuments()
db.security_logs.deleteOne(user:"alice",action:"logout")
db.security_logs.countDocuments()
```

Résultats observés :

```
15
  acknowledged: true, deletedCount: 1
14
```

Analyse. La suppression est techniquement démontrée : le nombre total de documents passe de 15 à 14. Néanmoins, le filtre utilisé reste ambigu, car le dataset initial contenait déjà un événement `alice/logout`. Ainsi, la commande prouve qu'un document de ce type a été supprimé, mais ne garantit pas formellement qu'il s'agissait du document ajouté en B1. Pour une suppression strictement traçable, il aurait fallu filtrer aussi sur l'horodatage ou sur l'`ObjectId` retourné par `insertMany`.



```
noseclab> db.security_logs.countDocuments()
15
noseclab> db.security_logs.deleteOne({user:"alice",action:"logout"})
{ acknowledged: true, deletedCount: 1 }
noseclab> db.security_logs.countDocuments()
14
```

FIGURE 5 – Contrôle avant et après suppression d'un document `alice/logout`.

B5. Vérification de l'authentification et analyse sécurité

Les vérifications observées sont les suivantes :

```
nmap -p 27017 localhost
curl -s http://localhost:27017
docker exec -it mongo-noseclab mongosh --eval "db.adminCommand( listDatabases: 1 )"
docker port mongo-noseclab
```

Résultats essentiels :

- 27017/tcp open mongod
- It looks like you are trying to access MongoDB over HTTP on the native driver port.
- liste complète des bases `admin`, `config`, `local` et `noseclab` sans mot de passe
- 27017/tcp -> 0.0.0.0:27017

Paragraphe d'analyse demandé. Le résultat est problématique car il montre qu'une personne non authentifiée peut non seulement détecter le service MongoDB, mais aussi s'y connecter et lister les bases de données sans fournir d'identifiants. Dans un contexte réel, cela signifie qu'un attaquant pourrait lire, modifier ou supprimer des journaux de sécurité, voire altérer des preuves d'incident. Le bind réseau sur 0.0.0.0 aggrave encore la situation puisqu'il rend le service potentiellement accessible depuis d'autres machines. Pour corriger cela, il faut activer l'authentification avec `-auth`, créer des comptes et rôles adaptés, et limiter l'exposition réseau en publiant le port uniquement sur 127.0.0.1 ou en le protégeant par des règles de filtrage réseau.

```

(Kali@DESKTOP-PNL51AU)-[~]
$ nmap -p 27017 localhost
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-05 11:07 +00
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000071s latency).

PORT      STATE SERVICE
27017/tcp  open  mongod

Nmap done: 1 IP address (1 host up) scanned in 0.14 seconds

(Kali@DESKTOP-PNL51AU)-[~]
$ curl -s http://localhost:27017
It looks like you are trying to access MongoDB over HTTP on the native driver port.

(Kali@DESKTOP-PNL51AU)-[~]
$ docker exec -it mongo-noseclab mongosh --eval "db.adminCommand({ listDatabases: 1 })"
{
  databases: [
    { name: 'admin', sizeOnDisk: Long('40960'), empty: false },
    { name: 'config', sizeOnDisk: Long('110592'), empty: false },
    { name: 'local', sizeOnDisk: Long('40960'), empty: false },
    { name: 'noseclab', sizeOnDisk: Long('73728'), empty: false }
  ],
  totalSize: Long('266240'),
  totalSizeMb: Long('0'),
  ok: 1
}

(Kali@DESKTOP-PNL51AU)-[~]
$ docker port mongo-noseclab
27017/tcp -> 0.0.0.0:27017
27017/tcp -> [::]:27017

```

FIGURE 6 – Vérifications de surface d’attaque : port ouvert, réponse du service, listage des bases sans mot de passe et bind réseau sur toutes les interfaces.

5. Évaluation synthétique du travail réalisé

Bloc	Constat	Statut
Déploiement Docker	Conteneur MongoDB lancé avec port exposé et volume persistant	Réussi
Connexion shell	Connexion à <code>mongosh</code> opérationnelle après correction des erreurs initiales	Réussi
CRUD	Insertion, lecture, mise à jour et suppression démontrées	Réussi
Filtrage avancé	Requête <code>\$in + \$gte</code> + projection + tri réalisée	Réussi
Mise à jour ciblée	Quatre documents modifiés pour l’IP 10.0.0.50	Réussi
Sécurité	Port ouvert, auth absente, bind réseau exposé explicitement démontrés	Réussi
Précision de modélisation	Quelques écarts de nommage ou de ciblage dans B1, B3 et B4	À améliorer

6. Conclusion

Le travail réalisé couvre correctement les objectifs fondamentaux de la séance : comprendre le modèle MongoDB, exécuter des opérations CRUD, formuler des requêtes d'investigation et identifier une configuration non sécurisée. Le point le plus important sur le plan cybersécurité est la démonstration concrète qu'une instance MongoDB exposée sur `0.0.0.0:27017` et démarrée sans authentification permet à un tiers de détecter le service et de lister les bases de données sans contrôle d'accès. Le TP atteint donc bien son objectif pédagogique : associer la prise en main technique de MongoDB à un raisonnement sécurité immédiatement applicable.